← Back

Programming Homework 2
Graded Assignment • 30 min

Due Nov 11, 11:59 PM IST

Your grade: 100%

Your latest: **100%** • Your highest: **100%** • To pass you need at least 70%. We keep your highest score.

Next item →

1. How many alignments does the naive exact matching algorithm try when matching the string `GGCGCGGTGGCTCACGCCTGTAATCCCAGCACTTTGGGAGGCCGAGG` (derived from human Alu sequences) to the excerpt of human chromosome 1? (Don't consider reverse complements.)

1 / 1 point

✓ Nice work

2. How many character comparisons does the naive exact matching algorithm try when matching the string `GGCGCGGTGGCTCACGCCTGTAATCCCAGCACTTTGGGAGGCCGAGG` (derived from human Alu sequences) to the excerpt of human chromosome 1? (Don't consider reverse complements.)

1 / 1 point

✓ Nice work

3. How many alignments does Boyer-Moore try when matching the string `GGCGCGGTGGCTCACGCCTGTAATCCCAGCACTTTGGGAGGCCGAGG` (derived from human Alu sequences) to the excerpt of human chromosome 1? (Don't consider reverse complements.)

1 / 1 point

✓ Nice work

4. **Index-assisted approximate matching.** In practicals, we built a Python class called `Index` implementing an ordered-list version of the k-mer index. The `Index` class is copied below.

1 / 1 point

```
class Index(object):
    def __init__(self, t, k):
        ''' Create index from all substrings of size 'length' '''
        self.k = k  # k-mer length (k)
        self.index = []
        for i in range(len(t) - k + 1):  # for each k-mer
            self.index.append((t[i:i+k], i))  # add (k-mer, offset) pair
        self.index.sort()  # alphabetize by k-mer

    def query(self, p):
        ''' Return index hits for first k-mer of P '''
        kmer = p[:self.k]  # query with first k-mer
        i = bisect.bisect_left(self.index, (kmer, -1))  # binary search
        hits = []
        while i < len(self.index):  # collect matching index entries
            if self.index[i][0] != kmer:
                break
            hits.append(self.index[i][1])
            i += 1
        return hits
```

We also implemented the pigeonhole principle using Boyer-Moore as our exact matching algorithm.

Implement the pigeonhole principle using `Index` to find exact matches for the partitions. Assume P always has length 24, and that we are looking for approximate matches with up to 2 mismatches (substitutions). We will use an 8-mer index.

Download the Python module for building a k-mer index.

https://d28rh4a8wq0iu5.cloudfront.net/ads1/code/kmer_index.py

Write a function that, given a length-24 pattern P and given an `Index` object built on 8-mers, finds all approximate occurrences of P within T with up to 2 mismatches. Insertions and deletions are not allowed. Don't consider any reverse complements.

How many times does the string `GGCGCGGTGGCTCACGCCTGTAAT`, which is derived from a human Alu sequence, occur with up to 2 substitutions in the excerpt of human chromosome 1? (Don't consider reverse complements here.)

Hint 1: Multiple index hits might direct you to the same match multiple times, but be careful not to count a match more than once.

Hint 2: You can check your work by comparing the output of your new function to that of the `naive_2mm` function implemented in the previous module.

✓ Nice work

5. Using the instructions given in Question 4, how many total index hits are there when searching for occurrences of `GGCGCGGTGGCTCACGCCTGTAAT` with up to 2 substitutions in the excerpt of human chromosome 1?

1 / 1 point

(Don't consider reverse complements.)

Hint: You should be able to use the `boyer_moore` function (or the slower `naive` function) to double-check your answer.